■ NLP Notebook

Understanding Every Cell — Input, Output & Theory

Quora Duplicate Question Detection

Beginner-Friendly Documentation

■ Big Picture — What This Notebook Does

This notebook solves a real-world NLP problem: **detecting duplicate questions on Quora**. The full pipeline flows as follows:

Step	What Happens	Output
1. Environment Setup	Import all libraries	Tools ready to use
2. Data Loading	Download Quora dataset from Hugging Face	DataFrame with 4 lakh rows
3. EDA	Inspect shape, types, sample rows	Understanding of data structure
4. Noise Analysis	Find & fix nulls and duplicates	Clean dataset
5. Label Distribution	Count duplicate vs non-duplicate pairs	Class balance chart
6. Text Length	Measure char & word length, spot outliers	Histograms & boxplots
7. NLP Pipeline	Tokenize, remove stopwords, clean text	Clean token lists
8. Sample Analysis	Browse actual duplicate & non-dup pairs	Human understanding
9. Baseline Model	TF-IDF + Logistic Regression + Evaluation	Accuracy, F1, Confusion Matrix

Section 1 — Environment Setup

Before any analysis begins, all required Python libraries must be imported. Think of this like gathering every tool before starting a lab experiment — if something is missing, the whole notebook fails.

Libraries and Their Roles

Library	Role
pandas	Works with tabular data — like Excel but in Python. Used for all DataFrame operations.
numpy	Math operations on arrays and large number sets.
matplotlib	Draws charts, histograms, bar graphs.
seaborn	Higher-level chart library built on matplotlib — makes prettier plots easily.
re	Regular Expressions — find and remove character patterns from text.
nltk	Natural Language Toolkit. Handles tokenization and stopword lists.
datasets	Hugging Face library to download publicly available ML datasets.
sklearn	Scikit-learn — the main ML library. Handles TF-IDF, model training, evaluation.
scipy.sparse	Handles very large but mostly-empty matrices efficiently (TF-IDF output).

What do the nltk.download() lines do?

NLTK needs small data files installed separately before it can work:

- **punkt** — teaches NLTK how to split sentences into individual words (tokenization rules).
- **punkt_tab** — updated tokenizer data for newer NLTK versions.
- **stopwords** — a pre-built list of common words like 'the', 'is', 'and' that carry no meaning.

■ *quiet=True suppresses the download confirmation messages so your notebook output stays clean.*

Expected Output

```
No errors.
Possibly 3 short NLTK download confirmation lines — that is completely normal.
```

Section 2 — Data Loading

This cell downloads the **Quora Duplicate Questions** dataset from Hugging Face and loads the training portion into a pandas DataFrame.

What is this dataset?

It contains pairs of questions scraped from Quora.com. For each pair, there is a label:

- **label = 0** — the two questions are NOT duplicates (they ask different things).
- **label = 1** — the two questions ARE duplicates (same meaning, different words).

Example rows

sentence1	sentence2	label
How do I learn Python?	What is the best way to learn Python?	1 (Duplicate)
What is the capital of France?	How do I cook pasta?	0 (Not Duplicate)

Expected Output

```
Dataset loaded: 404,290 rows, 3 columns
```

Section 3 — EDA: Basic Inspection

Four quick cells let you peek at your data before doing anything with it. Always do this first — never assume the data looks the way you expect.

Cell: df.head()

Shows the first 5 rows as a visual table. Confirms the data loaded correctly and lets you see what columns exist: sentence1, sentence2, label.

Expected Output

```
A 5-row HTML table with 3 columns: sentence1, sentence2, label.
```

Cell: df.shape

Returns (rows, columns) — tells you the total size of your dataset.

Expected Output

```
(404290, 3) → about 4 lakh question pairs with 3 columns.
```

Cell: df.describe()

Gives a statistical summary of numeric columns only. For this dataset only 'label' is numeric. You will see: count (total rows), mean (average label ≈ 0.37 means 37% are duplicates), min=0 and max=1 confirming labels are binary.

Expected Output

```
Statistical table with count, mean, std, min, 25%, 50%, 75%, max for label column.
```

Cell: df.info()

Shows column names, data types, and how many non-null values exist. Useful for spotting missing data at a glance.

Expected Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 404290 entries
# Column Non-Null Count Dtype
0 sentence1 404290 non-null object
1 sentence2 404290 non-null object
2 label 404290 non-null int64
```

Section 4 — Noise & Missing Value Analysis

Real-world datasets are messy. This section detects and fixes two common data quality issues before any analysis or modelling begins.

Cell 1 — Detecting Problems

isnull().sum() counts how many cells in each column are empty (NaN/None). If any sentence is missing, the model cannot process that row.

duplicated().sum() counts how many rows are exact copies of another row. Duplicates can bias model training — the model sees the same example multiple times and over-learns it.

Expected Output (if data is clean)

```
Null values per column:
sentence1 0
sentence2 0
label 0

Duplicate rows: 0
```

Cell 2 — Fixing the Problems

Even if this dataset is clean, this cell is a best practice to always include:

- **dropna()** — removes any row where sentence1 or sentence2 is empty.
- **drop_duplicates()** — removes fully identical rows.
- **reset_index()** — renumbers rows from 0 after deletions so index is continuous.

Expected Output

```
Shape after cleaning: (404290, 3) ← same if no nulls/duplicates existed
```

Section 5 — Label Distribution

This cell draws a bar chart showing how many pairs are duplicates vs non-duplicates, and prints the percentage breakdown.

Why does this matter?

In Machine Learning, if your data is heavily skewed — say 95% label=0 and 5% label=1 — the model could simply always predict 0 and appear 95% accurate without ever learning anything meaningful. This is called **class imbalance**.

Checking the distribution first tells you whether you need balancing techniques like oversampling or class weights.

Expected Output — Bar Chart

Two bars: one for label=0 (non-duplicate), one for label=1 (duplicate). Label=0 bar should be taller since most Quora questions are not duplicates.

Expected Output — Percentage Text

```
0 63.08%
1 36.92%
```

This means roughly 63% of pairs are non-duplicates and 37% are duplicates. A slight imbalance, but manageable — no special balancing technique is strictly required for a baseline model.

Section 6 — Text Length Analysis

Before modelling, you need to understand how long your questions are. Very short or extremely long texts can affect model performance and your choice of model architecture.

Cell 1 — Character Length

df['sentence1'].apply(len) counts every character — letters, spaces, punctuation — in each sentence.

The **.describe()** output gives you a statistical summary:

- **mean** ≈ 60–70 characters → an average question is about one sentence long.
- **max** could be 300+ → some questions are paragraph-length (outliers to watch out for).
- **std** (standard deviation) tells you how much lengths vary from the average.

Cell 2 — Length Histogram

An overlapping histogram for both sentences shows the **distribution** of lengths visually.

How to read it:

- A **tall bar on the left** means most questions are short — typical for Quora.
- A **long tail to the right** means a few questions are very long (outliers).
- If both sentences have **similar shapes**, they are comparable in length — good sign.

Cell 3 — Word Count

Instead of characters, this counts **words** using **x.split()** which splits on spaces.

Example: **'How are you'** → **['How', 'are', 'you']** → length = 3

Expected Output

```
que1_words que2_words
mean 10.84 10.61
max 237.00 237.00
```

Average question has about 10 words. Max 237 is an outlier — possibly a paragraph-length Quora post.

Cell 4 — Boxplot (Outlier Detection)

A boxplot is a visual summary of spread and outliers. Here is how to read one:

```
|----[ box ]---| o o o
^ ^ ^ ^
min 25% 75% max (dots = outliers)
```

- The **box** covers the middle 50% of data.
- The **line inside** the box is the median (middle value).
- **Dots beyond the whiskers** are outliers — unusually long questions.

If you see many dots far to the right, those are very long questions that might confuse your model or need to be truncated.

Section 7 — NLP Pipeline

This is the heart of text preprocessing. Raw text cannot go into an ML model directly — it needs to be cleaned, structured, and eventually converted into numbers. This section handles the text side of that preparation.

7a — Tokenization

What is Tokenization?

Tokenization is the process of breaking a sentence into individual units called **tokens** (usually words or punctuation marks).

Example

```
Input : "How do I learn Python?"
Output: ['How', 'do', 'I', 'learn', 'Python', '?']
```

Why use NLTK's word_tokenize instead of just split()?

x.split() blindly splits on spaces. It treats **"don't"** as one token.

word_tokenize is smarter — it handles contractions, punctuation, and edge cases correctly:

```
split() : ["don't"] — wrong, 1 token
word_tokenize() : ['do', "n't"] — correct, 2 tokens
```

Expected Output

```
Sample tokens (sentence1):
['what', 'is', 'the', 'step', 'by', 'step', 'guide', 'to',
'invest', 'in', 'share', 'market', '?']
```

7b — Most Common Words

What are Stopwords?

Stopwords are common English words that appear in almost every sentence but carry no meaningful information about the topic:

```
Examples: 'the', 'is', 'a', 'in', 'and', 'of', 'to', 'it', 'that'
```

Removing them helps focus on words that actually distinguish duplicate from non-duplicate pairs.

What this cell does

- Flattens all tokens from sentence1 into one big list.
- Filters out stopwords and non-alphabetic tokens (punctuation, numbers).
- Counts word frequency using **Counter**.
- Plots the top 20 most common meaningful words as a bar chart.

Expected Output — Chart

A bar chart showing the 20 most frequent words after stopword removal. You will likely see words like: **best, way, difference, make, use, get, world, life** — common Quora question vocabulary.

> ■ *These common words are informative for understanding your dataset but won't be very useful features for the model — TF-IDF handles that by down-weighting them automatically.*

7c — Text Cleaning

What the clean_text() function does

This function takes raw text and returns a cleaned version in two steps:

- **Step 1 — Lowercase:** Converts all characters to lowercase so 'Python' and 'python' are treated as the same word. Without this, the model would see them as different words.
- **Step 2 — Remove special characters:** Strips everything except letters, numbers, and spaces. Removes ?, !, /, @, #, $, brackets, etc.

Example

```
Input : "How do I learn Python? It's amazing!"
Output: "how do i learn python its amazing"
```

Expected Output — Table

Side-by-side comparison of original vs cleaned text for both sentences. You should see: lowercase, no punctuation, same words preserved.

Section 8 — Sample Analysis

Before training any model, it is important to read actual examples from your dataset. This builds intuition about how hard the problem is and what patterns the model needs to learn.

Duplicate Pairs Cell (label = 1)

Filters rows where label=1 and displays 5 examples.

What to observe: The two questions should mean the same thing but be worded differently. Sometimes the difference is very subtle — just word order or synonyms.

Example Duplicate Pair

sentence1	sentence2
How can I lose weight fast?	What is the quickest way to reduce weight?

Non-Duplicate Pairs Cell (label = 0)

Filters rows where label=0 and displays 5 examples.

What to observe: Questions might share some words but ask completely different things. This is what makes the problem hard — the model must understand meaning, not just word overlap.

Example Non-Duplicate Pair

sentence1	sentence2
How do I invest in shares?	What would happen if India went to war?

■ *Doing this manual review is a habit every data scientist follows. It often reveals labelling errors, weird edge cases, or domain-specific patterns you would never spot just from statistics.*

Section 9 — Baseline Model

A **baseline model** is the simplest possible model you can build. Its purpose is to set a performance benchmark. Every future, more complex model (BERT, LSTM, etc.) should beat this baseline. If it doesn't, something is wrong with the complex model.

9a — TF-IDF Vectorization

Why can't we feed raw text to a model?

Machine learning models only understand numbers. Text must be converted to a numerical representation. TF-IDF is one of the most common ways to do this.

What is TF-IDF?

TF-IDF stands for **Term Frequency — Inverse Document Frequency**. It converts each sentence into a vector (list of numbers), where each number represents how important a specific word is in that sentence.

- **TF (Term Frequency)** — how often a word appears in this particular sentence. If 'python' appears 3 times in a sentence of 10 words, TF = 3/10 = 0.3.
- **IDF (Inverse Document Frequency)** — how rare the word is across ALL sentences. Rare words get higher scores. 'python' in a tech question gets a high IDF. 'the' gets near zero.
- **TF-IDF score** = TF × IDF. High score means the word is frequent in THIS sentence but rare overall — making it a strong signal.

Settings Explained

Parameter	Value	Meaning
max_features	5000	Only keep the 5000 most useful words — ignores very rare words.
stop_words	'english'	Automatically remove common English stopwords.
ngram_range	(1,2)	Consider both single words ('learn') AND word pairs ('learn Python') as features.

Why fit_transform for q1 but transform for q2?

fit_transform does two things: **learns** the vocabulary from the data, then **converts** it. You only ever let the model learn from training data (q1 here).

transform only converts — using the vocabulary already learned. You never let the model learn anything new from q2 or test data. This prevents **data leakage**.

Expected Output

```
Feature matrix shape: (404290, 10000)
← 10000 because two 5000-feature vectors are stacked side by side
```

9b — Train-Test Split

The dataset is divided into two parts:

- **80% Training set** — the model learns from this data. It sees the questions and their labels.
- **20% Test set** — the model is evaluated on this data. It has never seen these rows during training.

This separation is critical. If you evaluate on the same data you trained on, the model appears perfect — it just memorised the answers. The test set simulates real-world unseen data.

What does stratify=y do?

It ensures the 63/37 label ratio is preserved in both splits. Without it, by random chance, your test set might be all non-duplicates — making evaluation meaningless.

Expected Output

```
Train size: 323,432 | Test size: 80,858
```

9c — Model Training

What is Logistic Regression?

Despite the name, Logistic Regression is used for **classification**, not regression. It learns a mathematical boundary (a line/plane in high-dimensional space) that separates duplicate pairs from non-duplicate pairs.

It looks at the TF-IDF features and asks: which words (or word pairs) tend to appear together in duplicate questions? Those patterns become the decision boundary.

max_iter=1000 means the algorithm tries up to 1000 rounds of optimization to find the best possible boundary. More iterations = more time to converge for large datasets.

Expected Output

```
Model training complete.
```

9d — Evaluation

Accuracy

The percentage of total predictions that were correct.

```
Accuracy = (Correct Predictions) / (Total Predictions)
```

```
Accuracy Score: 0.78 → 78% of pairs were correctly classified
```

Accuracy alone can be misleading with imbalanced classes — which is why we also check F1.

F1 Score

F1 balances two things:

- **Precision** — of all pairs predicted as duplicate, how many actually were? (Avoids false alarms.)
- **Recall** — of all actual duplicate pairs, how many did the model catch? (Avoids missed duplicates.)

F1 = 2 × (Precision × Recall) / (Precision + Recall) — a single number capturing both.

Classification Report

Gives precision, recall, and F1 for each label (0 and 1) separately. Pay special attention to the **row for label=1** — that tells you how well the model detects actual duplicates, which is the harder and more important task.

9e — Confusion Matrix

What is a Confusion Matrix?

A 2×2 table that shows exactly where your model got confused:

	Predicted Non-Duplicate	Predicted Duplicate
Actual Non-Duplicate	True Negative (TN) ■	False Positive (FP) ■
Actual Duplicate	False Negative (FN) ■	True Positive (TP) ■

- **TN** — correctly said 'not duplicate'. Model got it right.
- **TP** — correctly said 'duplicate'. Model got it right.
- **FP** — said 'duplicate' but it wasn't. False alarm — annoying but tolerable.
- **FN** — said 'not duplicate' but it was. Missed a duplicate — worse for this task.

How to read the heatmap

The chart uses color intensity — **darker blue = higher count**. A good model has the two diagonal cells (TN and TP) darkest, and the off-diagonal cells (FP and FN) as light as possible.

9f — Sample Predictions

Shows 10 actual rows with the model's prediction next to the true label. This is a human-readable sanity check.

label	Predicted	Meaning
1	1	Model correctly identified a duplicate ■
0	0	Model correctly identified a non-duplicate ■
1	0	Model missed a duplicate (False Negative) ■
0	1	Model falsely flagged a non-duplicate (False Positive) ■

Go back and read the actual question text for the wrong predictions. This gives you intuition for why the model failed and what improvements to make next.

What's Next — Beyond the Baseline

Your baseline gives you a reference score. The real learning begins when you try to beat it. Here are the natural next steps in increasing order of complexity:

Next Step	Technique	Expected Improvement
Better features	Add shared word count, Jaccard similarity, length difference as features	Small to medium
Better text cleaning	Lemmatization, stemming, remove numbers	Small
Word Embeddings	Word2Vec or GloVe to capture semantic meaning	Medium
Sentence Embeddings	Sentence-BERT (SBERT) — state-of-the-art for sentence similarity	Large
Deep Learning	Siamese LSTM or Transformer-based models	Large

■ *Every improvement should be compared against your baseline F1 score. If a complex model does worse than Logistic Regression, you know something went wrong in preprocessing or training.*

End of Documentation